

# Grouped-Query Attention vs. Standard Multi-Head Attention: A Controlled Comparison at Small Scale

Kapil

August 2026

## Abstract

We present a controlled empirical comparison between grouped-query attention (GQA) and standard multi-head attention (MHA) in a small ( $\sim 13.6$ M parameter) GPT-style decoder-only Transformer, trained from scratch on the TinyStories dataset. Holding architecture, random seed, data, and training budget (30,000 steps) fixed, and varying only the number of key/value heads (2 for GQA vs. 4 for MHA, with 4 query heads throughout), we find the two configurations reach nearly identical validation loss (2.2500 for GQA vs. 2.2451 for MHA; perplexity 9.49 vs. 9.44) while GQA uses exactly half the key/value projection parameters (65,536 vs. 131,072). This small, controlled result is consistent with the finding reported for GQA at much larger scale [2]: the technique gives a real parameter and memory saving at negligible cost to model quality, and this cost remains negligible even at a scale three-plus orders of magnitude smaller than where GQA is typically studied.

## 1 Motivation

The GPT implementation underlying this study was built entirely from scratch in PyTorch causal self-attention, grouped-query attention, Transformer decoder blocks, and the training loop were all hand-written rather than drawn from a pretrained model or an existing Transformer library. That implementation uses grouped-query attention because it is a well-known efficiency technique in modern large language models (LLaMA, Mistral, and others), not because its cost/benefit tradeoff had been verified independently at small scale. This experiment isolates that one architectural choice and measures it directly, rather than assuming the large-scale result transfers unchanged.

**Research question:** at a fixed, small parameter budget and fixed training budget, how much does reducing the number of key/value heads (GQA) cost in validation loss compared to standard multi-head attention, and how much does it save in key/value parameter count?

## 2 Background

### 2.1 Causal Self-Attention

Standard causal self-attention computes, for query, key, and value projections  $Q$ ,  $K$ ,  $V$  of an input sequence  $X$ :

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (1)$$

where  $M$  is a causal mask preventing position  $i$  from attending to positions  $j > i$ , and  $d_k$  is the per-head dimension. In multi-head attention, this is computed independently across  $h$  heads and the outputs concatenated and projected back to the model dimension.

## 2.2 Grouped-Query Attention

Grouped-query attention [2] reduces the number of distinct key/value projections relative to query projections. Given  $n_q$  query heads and  $n_{kv}$  key/value heads with  $n_q \bmod n_{kv} = 0$ , each key/value head is shared by  $n_q/n_{kv}$  query heads. Standard MHA is the special case  $n_{kv} = n_q$ ; multi-query attention (MQA) is the special case  $n_{kv} = 1$ . GQA interpolates between these, trading some of MQA’s aggressive parameter/cache reduction for MHA’s representational flexibility.

The primary motivation for GQA is not training-time parameter count but *inference-time* memory: the key/value tensors for all previously generated tokens must be cached during autoregressive generation, and this cache’s size scales with  $n_{kv}$ , not  $n_q$ . Reducing  $n_{kv}$  directly reduces this cache, which matters increasingly at longer context lengths and larger batch sizes.

## 3 Experimental Setup

Both configurations share an identical architecture and training procedure, differing only in  $n_{kv}$  (Table 1).

Table 1: Attention configuration comparison.

|                       | GQA | MHA              |
|-----------------------|-----|------------------|
| Query heads ( $n_q$ ) | 4   | 4                |
| KV heads ( $n_{kv}$ ) | 2   | 4                |
| KV group size         | 2   | 1 (standard MHA) |

Shared hyperparameters are given in Table 2. Both models were trained sequentially within the same script, with `torch.manual_seed(1337)` set immediately before model construction in both cases, so both models start from comparable initialization conditions and observe training data in the same order.

Table 2: Shared model and training configuration.

| Hyperparameter                      | Value                                                        |
|-------------------------------------|--------------------------------------------------------------|
| Embedding dimension ( $d_{model}$ ) | 128                                                          |
| Transformer layers                  | 4                                                            |
| Context length                      | 128                                                          |
| Feed-forward dimension              | 512                                                          |
| Vocabulary (GPT-2 BPE)              | 50,257                                                       |
| Dropout                             | 0.1                                                          |
| Batch size                          | 32                                                           |
| Optimizer                           | AdamW ( $\text{lr} = 3 \times 10^{-4}$ , weight decay = 0.1) |
| Training steps                      | 30,000                                                       |
| Random seed                         | 1337 (identical for both runs)                               |
| Dataset                             | 500,000 TinyStories stories, 90/10 train/val split           |
| Hardware                            | 1 × GPU (Kaggle, T4-class)                                   |

## 4 Results

### 4.1 Validation Loss and Perplexity

Table 3 reports final validation loss and perplexity at step 30,000 for both configurations.

Table 3: Final validation loss and perplexity at step 30,000.

| Configuration          | KV heads | Val. loss | Val. perplexity |
|------------------------|----------|-----------|-----------------|
| GQA                    | 2        | 2.2500    | 9.49            |
| MHA                    | 4        | 2.2451    | 9.44            |
| Difference (MHA – GQA) |          | −0.0049   | −0.05           |

The two configurations converge to nearly identical validation loss. The gap (0.0049 nats, well under 1% relative difference in perplexity) is small enough that, without repeated-seed runs, it cannot be confidently distinguished from run-to-run noise, but it is clearly not a large or practically meaningful quality cost.

### 4.2 Training Dynamics

Both models exhibit smooth, monotonic loss decreases with no signs of instability, and validation loss tracks training loss closely throughout for both configurations, indicating neither model overfits within this training budget:

- **GQA:** validation loss 6.04 (step 250) → 2.90 (step 8,750) → 2.25 (step 30,000)
- **MHA:** validation loss 6.04 (step 250) → 2.79 (step 8,750) → 2.245 (step 30,000)

The two curves are visually near-indistinguishable across the full 30,000-step run.

### 4.3 Parameter Cost

Table 4: Key/value projection and total parameter counts.

| Configuration         | KV projection params | Total params            |
|-----------------------|----------------------|-------------------------|
| GQA                   | 65,536               | 13,607,936              |
| MHA                   | 131,072              | 13,673,472              |
| Savings (GQA vs. MHA) | 65,536 (50%)         | 65,536 (0.48% of total) |

GQA exactly halves the key/value projection parameters, as expected (2 KV heads vs. 4, everything else identical). At this small model scale, this is a modest 0.48% reduction in *total* parameters, because KV projections are a small fraction of a model whose parameter count is dominated by the token embedding and language-modeling head (together  $\sim 94\%$  of total parameters at  $d_{model} = 128$  with a 50,257-token vocabulary). The practical benefit of GQA is understated by this total-parameter view: its larger real-world payoff is in KV-cache memory during autoregressive inference, which scales with KV projection size, not total parameter count, and matters increasingly at longer context lengths and larger batch sizes than used in this study.

## 5 Discussion

### 5.1 Interpretation

At this small scale (13.6M parameters, 128-token context, 4 layers), halving the number of key/value heads via GQA produces no meaningfully detectable quality cost relative to standard multi-head attention, while providing an exact, verifiable halving of key/value parameters. This corroborates independently, at a scale roughly three orders of magnitude smaller than typically studied the core claim of the original GQA work [2]: quality is preserved while attention’s key/value footprint shrinks.

### 5.2 Limitations

- **Single seed.** Results are from one training run per configuration. We report the observed difference (0.0049 loss) but do not have a variance estimate across seeds, so we cannot rule out that a different seed could show a somewhat larger or smaller gap. The difference observed is small enough that it may not be statistically distinguishable from seed-to-seed noise.
- **Single scale.** This study covers exactly one model size and one GQA group size (2:1). It does not establish how the GQA/MHA gap changes at larger scale, longer context, or different group ratios (e.g. 4:1, 8:1).
- **A third configuration** (weight-tied token embedding and output projection) was planned but not completed, due to repeated training-session interruptions on the compute platform used; this report is scoped to the GQA-vs-MHA comparison only.
- **Generation-speed comparison was not completed** for the same reason; this report presents the parameter-count and loss comparison only, not an inference-latency measurement.

### 5.3 Future Work

A natural extension is repeating this comparison across multiple seeds to establish whether the observed 0.0049 loss gap is statistically meaningful, and across multiple GQA group sizes (e.g. 4:1, 8:1) to characterize how the quality/parameter tradeoff curve behaves as key/value sharing increases. Measuring actual inference latency and KV-cache memory at longer context lengths, where GQA’s benefit is expected to be most visible, would also strengthen the practical case made here.

## 6 Reproducibility

All code for this experiment is self-contained and independent of any pretrained model or third-party Transformer implementation. To reproduce:

```
pip install -r requirements.txt
python prepare_tinystories.py --output data/tinystories.txt --num-stories 500000
python run_experiment.py --data-path data/tinystories.txt --max-steps 30000
```

Both configurations use `seed=1337`, fixed via `torch.manual_seed()` immediately before model construction.

## References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. arXiv preprint arXiv:2305.13245, 2023.
- [3] Ronen Eldan and Yuanzhi Li. *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* arXiv preprint arXiv:2305.07759, 2023.